

Analytical Problem Solving

Warehouse Robot Navigation using BFS and DFS

SOLUTION:

Problem Set-up and Assumptions

The warehouse floor is modelled as a 5×5 grid. Each cell is written as (row, column) with both numbers running from 1 to 5, starting at the top-left corner. The delivery robot begins at (1,1) and has to reach the drop-off point (5,5). The shelving units occupy the cells $B = \{(1,3), (2,5), (4,4), (5,2), (5,3)\}$, which leaves $25 - 5 = 20$ free cells.

r \ c	1	2	3	4	5
1	START (1,1)	(1,2)	X	(1,4)	(1,5)
2	(2,1)	(2,2)	(2,3)	(2,4)	X
3	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
4	(4,1)	(4,2)	(4,3)	X	(4,5)
5	(5,1)	X	X	(5,4)	GOAL (5,5)

- **Grid key:** X marks a cell occupied by a shelving unit (blocked).
- **Moves:** Down = (r+1, c), Right = (r, c+1), Up = (r-1, c), Left = (r, c-1). Diagonal moves are not allowed and every move costs 1.
- **Tie-breaking:** neighbours are always examined in the fixed order **Down, Right, Up, Left**, so every trace below can be reproduced exactly.
- **Graph search:** BFS and DFS both remember the cells already reached, so no cell is put on the frontier twice.
- **Goal test:** BFS checks for the goal when a node is generated; DFS checks when a node is popped (expanded).
- The robot is already carrying the package at the start and the warehouse layout is known in advance.

Part (a): PEAS Description of the Warehouse Robot

PEAS	Description for this warehouse robot
Performance measure	Deliver the package to cell (5,5) safely and without damage. Use as few moves as possible; the best possible cost on this layout is 8 moves. Avoid collisions with shelves and avoid revisiting cells or making wasted moves. Keep travel time and battery consumption low.
Environment	A 5×5 warehouse floor with 25 cells: 20 free cells and 5 shelf cells that cannot be entered. Start (1,1), goal (5,5); movement is only along rows and columns. Nature of the environment: fully observable (map is known), single agent, deterministic, static (shelves never move), discrete, sequential.
Actuators	Wheel/drive motors that carry the robot one cell Down, Right, Up or Left. Steering and braking unit so it can stop exactly in a cell. A tray or gripper that releases the package at the goal.
Sensors	Wheel encoders and floor RFID/QR tags to know the current cell. Ultrasonic, LIDAR or bumper sensors to detect shelves. A camera to recognise the goal marker, a load sensor to confirm the package is on board, and a battery gauge.

Part (b): Agent Type Classification

Best choice: a goal-based agent, i.e. an agent that holds a model of the grid and plans a sequence of actions that ends in the goal state.

Agent type	Suitable?	Reasoning for this warehouse
Simple reflex	No	It reacts only to what it senses at the moment. A rule such as "go Down or Right" breaks down at (1,2), where Right is blocked by the shelf at (1,3), and at (5,1), where Right is blocked by (5,2). The pocket at (5,4) is walled in by (4,4) and (5,3), so such an agent can get stuck or wander in circles.
Model-based reflex	Partly	Remembering its position and the map lets it avoid loops, but it still just applies condition-action rules. It has no idea where it is supposed to end up, so it cannot choose a route.
Goal-based	Yes	There is a single, clearly defined goal cell (5,5). The agent has to work out which chain of moves reaches it, which is precisely a search problem. Because the world is known, deterministic and static, the full path can be planned beforehand and then executed.
Utility-based	Not needed	Utility matters when several objectives compete or outcomes differ in quality (speed against safety against battery). Here there is one objective and every step costs the same, so path length alone is enough. It would become useful if some cells were slower or riskier.

Justification: the shelves create dead ends and walled pockets, so the robot has to look ahead instead of reacting blindly. The goal can be tested directly, and with equal step costs the whole task reduces to finding a goal-reaching, preferably shortest, sequence of moves. A goal-based agent with a map of the grid and a search algorithm matches this requirement.

Part (c): Formal Search Problem Formulation

Component	Definition
State space	$S = \{ (r, c) : 1 \leq r, c \leq 5 \text{ and } (r, c) \notin B \}$, giving 20 states.
Initial state	$s_0 = (1,1)$
Goal state / test	Goal = (5,5). GoalTest(s) is true exactly when $s = (5,5)$.
Actions	ACTIONS(r, c) = those moves among {Down, Right, Up, Left} that land inside the grid on a cell not in B. Example: ACTIONS(1,1) = {Down, Right}; ACTIONS(5,1) = {Up}.
Transition model	RESULT((r,c), Down) = (r+1, c) RESULT((r,c), Right) = (r, c+1) RESULT((r,c), Up) = (r-1, c) RESULT((r,c), Left) = (r, c-1) (each is defined only if the resulting cell is a legal free cell)
Path cost	Step cost $c(s, a, s') = 1$ for every action, so the path cost $g(n)$ equals the number of moves taken.
Solution	An action sequence leading from (1,1) to (5,5); it is optimal when its cost is the smallest possible. The Manhattan-distance lower bound is $ 5-1 + 5-1 = 8$, and a path of that length exists, so the optimal cost is 8.

Part (d): Breadth-First Search (BFS)

BFS keeps a FIFO queue and always expands the shallowest unexpanded node. The queue starts as [(1,1)]. The last column shows the queue from front to back after each expansion.

Step	Expand	New nodes added (D, R, U, L order)	Queue after step (front → back)
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(1,2), (3,1), (2,2)
3	(1,2)	none ((2,2) already in queue)	(3,1), (2,2)
4	(3,1)	(4,1), (3,2)	(2,2), (4,1), (3,2)
5	(2,2)	(2,3)	(4,1), (3,2), (2,3)
6	(4,1)	(5,1), (4,2)	(3,2), (2,3), (5,1), (4,2)

Step	Expand	New nodes added (D, R, U, L order)	Queue after step (front → back)
7	(3,2)	(3,3)	(2,3), (5,1), (4,2), (3,3)
8	(2,3)	(2,4)	(5,1), (4,2), (3,3), (2,4)
9	(5,1)	none (dead end)	(4,2), (3,3), (2,4)
10	(4,2)	(4,3)	(3,3), (2,4), (4,3)
11	(3,3)	(3,4)	(2,4), (4,3), (3,4)
12	(2,4)	(1,4)	(4,3), (3,4), (1,4)
13	(4,3)	none (all neighbours blocked or visited)	(3,4), (1,4)
14	(3,4)	(3,5)	(1,4), (3,5)
15	(1,4)	(1,5)	(3,5), (1,5)
16	(3,5)	(4,5)	(1,5), (4,5)
17	(1,5)	none ((1,4) visited, (2,5) blocked)	(4,5)
18	(4,5)	(5,5) = GOAL generated	(5,5)

Path reconstruction (via parent pointers): $(5,5) \leftarrow (4,5) \leftarrow (3,5) \leftarrow (3,4) \leftarrow (3,3) \leftarrow (3,2) \leftarrow (3,1) \leftarrow (2,1) \leftarrow (1,1)$.

Final path: $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$

Actions: Down, Down, Right, Right, Right, Right, Down, Down. **Total cost = 8.**

- 18 nodes were expanded and 19 of the 20 free cells were generated; cell (5,4) was never reached because it can only be entered from the goal itself. The queue never held more than 4 nodes.
- The cost of 8 matches the Manhattan lower bound, so the path is optimal. Other cost-8 routes exist (for instance $(1,1) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$), but this neighbour order reaches the left-then-middle-row route first.
- Had the goal test been applied on expansion rather than on generation, (5,5) would have been popped at step 19 with the same path and cost.

Part (e): Depth-First Search (DFS)

DFS keeps a LIFO stack. Neighbours are pushed in the reverse order (Left, Up, Right, Down) so that Down comes off the stack first. The stack starts as [(1,1)]. The last column shows the stack from top to bottom after each step.

Step	Pop / expand	Pushed (top first)	Stack after step (top → bottom)
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(3,1), (2,2), (1,2)
3	(3,1)	(4,1), (3,2)	(4,1), (3,2), (2,2), (1,2)
4	(4,1)	(5,1), (4,2)	(5,1), (4,2), (3,2), (2,2), (1,2)
5	(5,1)	none (dead end, backtrack)	(4,2), (3,2), (2,2), (1,2)
6	(4,2)	(4,3)	(4,3), (3,2), (2,2), (1,2)
7	(4,3)	(3,3)	(3,3), (3,2), (2,2), (1,2)
8	(3,3)	(3,4), (2,3)	(3,4), (2,3), (3,2), (2,2), (1,2)
9	(3,4)	(3,5), (2,4)	(3,5), (2,4), (2,3), (3,2), (2,2), (1,2)
10	(3,5)	(4,5)	(4,5), (2,4), (2,3), (3,2), (2,2), (1,2)
11	(4,5)	(5,5)	(5,5), (2,4), (2,3), (3,2), (2,2), (1,2)
12	(5,5)	(5,5) = GOAL, test passed	(2,4), (2,3), (3,2), (2,2), (1,2)

Path reconstruction: (5,5) ← (4,5) ← (3,5) ← (3,4) ← (3,3) ← (4,3) ← (4,2) ← (4,1) ← (3,1) ← (2,1) ← (1,1).

Final path: (1,1) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (3,3) → (3,4) → (3,5) → (4,5) → (5,5)

Actions: Down, Down, Down, Right, Right, Up, Right, Right, Down, Down. **Total cost = 10.**

- 12 nodes were expanded (11 non-goal nodes plus the goal). The stack never held more than 6 nodes, and (2,4), (2,3), (3,2), (2,2) and (1,2) were left unexplored when the goal was found.
- The only backtrack occurred at the dead end (5,1) in step 5. The route also contains an Up move at (4,3) → (3,3), which is a detour away from the goal.
- The path costs 10, which is **2 more than the optimum of 8**. DFS simply followed the first branch it met (Down along the left edge) instead of the shortest one. With the order Right, Up, Left, Down it would instead return

$(1,1) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$ with cost 8, showing that its result depends on the neighbour ordering.

Part (f): Comparison of BFS and DFS

Notation: b = branching factor (at most 4, and at most 3 once the parent is excluded), d = depth of the shallowest goal (8), m = maximum path depth (at most 19, since there are 20 free cells), $|V| = 20$ states.

Criterion	BFS	DFS
Completeness	Complete: the state space is finite and the shallowest goal lies at the finite depth $d = 8$, so BFS is guaranteed to find it.	Complete here only because a visited set is used on a finite graph. Plain tree-search DFS could loop for ever, since every move can be undone (Down/Up, Right/Left).
Optimality	Optimal: all step costs are equal, so the shallowest goal is also the cheapest. It returned cost 8, equal to the lower bound.	Not optimal in general. In this layout it returned cost 10, two moves worse than BFS (Part e).
Time complexity	$O(b^d)$. In the worst case for tree search this is about $4^8 = 65,536$ nodes; with a visited set it is $O(V + E)$. Observed: 18 expansions.	$O(b^m)$. With a visited set it is also $O(V + E)$. Observed: 12 expansions, fewer than BFS but along a longer route.
Space complexity	$O(b^d)$, because the entire frontier is stored. Observed peak queue size 4, plus the visited set of up to 20 cells.	$O(b \cdot m)$: only the current path and its unexplored siblings are stored. Observed peak stack size 6, plus the visited set.

BFS is the more suitable algorithm for this robot. Delivery time and battery use depend on the number of moves, and every move costs the same, so BFS's guarantee of the fewest moves is exactly what is needed. DFS expanded fewer nodes (12 against 18), but it delivered a longer 10-move route, and its answer can change whenever the neighbour order changes. The grid is tiny, so BFS's bigger frontier is not a problem. For a much larger warehouse an informed method such as A* with a Manhattan-distance heuristic would be better still.

Appendix: Python Code and Output

PYTHON CODE

```
"""
Warehouse delivery robot: BFS and DFS on a 5x5 grid.
Cells are (row, col), 1-indexed. Neighbour order everywhere: Down, Right, Up, Left.
"""
from collections import deque

N = 5
START, GOAL = (1, 1), (5, 5)
BLOCKED = {(1, 3), (2, 5), (4, 4), (5, 2), (5, 3)}

# action name -> (d_row, d_col); dict order = expansion order
ACTIONS = {"Down": (1, 0), "Right": (0, 1), "Up": (-1, 0), "Left": (0, -1)}

def successors(state):
    """Legal (action, next_state) pairs from a state."""
    r, c = state
    out = []
    for name, (dr, dc) in ACTIONS.items():
        nr, nc = r + dr, c + dc
        if 1 <= nr <= N and 1 <= nc <= N and (nr, nc) not in BLOCKED:
            out.append((name, (nr, nc)))
    return out

def trace_path(parent, state):
    path = []
    while state is not None:
        path.append(state)
        state = parent[state]
    return path[::-1]

def fmt(cells):
    return ", ".join(f"({r},{c})" for r, c in cells) if cells else "-"

def show(step, node, added, frontier):
    print(f"{step:>3} | {fmt([node]):<7} | {added:<22} | {fmt(frontier)}")

def bfs():
    frontier = deque([START])          # FIFO queue
    parent = {START: None}             # doubles as the visited set
    step, expanded, max_q = 0, 0, 1
    print("BFS TRACE")
    while frontier:
        node = frontier.popleft()
        step += 1
        expanded += 1
        added = []
        for _, nxt in successors(node):
            if nxt not in parent:
                parent[nxt] = node
                added.append(nxt)
            if nxt == GOAL:             # goal test on generation
                frontier.extend(added)
                show(step, node, fmt(added), list(frontier))
                return trace_path(parent, GOAL), expanded, max_q, len(parent)
        frontier.extend(added)
        max_q = max(max_q, len(frontier))
        show(step, node, fmt(added), list(frontier))
    return None, expanded, max_q, len(parent)
```

```

def dfs():
    stack = [START]                # LIFO stack
    parent = {START: None}
    seen = {START}
    step, expanded, max_s = 0, 0, 1
    print("DFS TRACE")
    while stack:
        node = stack.pop()
        step += 1
        expanded += 1
        if node == GOAL:            # goal test on expansion
            show(step, node, "GOAL REACHED", stack[::-1])
            return trace_path(parent, GOAL), expanded, max_s, len(seen)
        pushed = []
        for _, nxt in reversed(successors(node)):  # reversed so Down is popped first
            if nxt not in seen:
                seen.add(nxt)
                parent[nxt] = node
                stack.append(nxt)
                pushed.append(nxt)
        max_s = max(max_s, len(stack))
        show(step, node, fmt(pushed[::-1]), stack[::-1])
    return None, expanded, max_s, len(seen)

def actions_of(path):
    names = {(1, 0): "Down", (0, 1): "Right", (-1, 0): "Up", (0, -1): "Left"}
    return [names[(b[0] - a[0], b[1] - a[1])] for a, b in zip(path, path[1:])]

if __name__ == "__main__":
    for label, algo in (("BFS", bfs), ("DFS", dfs)):
        path, exp, peak, seen = algo()
        print(f"\n{label} path : {fmt(path)}")
        print(f"{label} moves : {' '.join(actions_of(path))}")
        print(f"{label} cost : {len(path) - 1}   expanded: {exp}   peak: {peak}\n")

```

PROGRAM OUTPUT:

BFS TRACE

1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(1,2), (3,1), (2,2)
3	(1,2)	-	(3,1), (2,2)
4	(3,1)	(4,1), (3,2)	(2,2), (4,1), (3,2)
5	(2,2)	(2,3)	(4,1), (3,2), (2,3)
6	(4,1)	(5,1), (4,2)	(3,2), (2,3), (5,1), (4,2)
7	(3,2)	(3,3)	(2,3), (5,1), (4,2), (3,3)
8	(2,3)	(2,4)	(5,1), (4,2), (3,3), (2,4)
9	(5,1)	-	(4,2), (3,3), (2,4)
10	(4,2)	(4,3)	(3,3), (2,4), (4,3)
11	(3,3)	(3,4)	(2,4), (4,3), (3,4)
12	(2,4)	(1,4)	(4,3), (3,4), (1,4)
13	(4,3)	-	(3,4), (1,4)
14	(3,4)	(3,5)	(1,4), (3,5)
15	(1,4)	(1,5)	(3,5), (1,5)
16	(3,5)	(4,5)	(1,5), (4,5)
17	(1,5)	-	(4,5)
18	(4,5)	(5,5)	(5,5)

BFS path : (1,1), (2,1), (3,1), (3,2), (3,3), (3,4), (3,5), (4,5), (5,5)

BFS moves : Down, Down, Right, Right, Right, Right, Down, Down

BFS cost : 8 expanded: 18 peak: 4

DFS TRACE

1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(3,1), (2,2), (1,2)
3	(3,1)	(4,1), (3,2)	(4,1), (3,2), (2,2), (1,2)
4	(4,1)	(5,1), (4,2)	(5,1), (4,2), (3,2), (2,2), (1,2)
5	(5,1)	-	(4,2), (3,2), (2,2), (1,2)
6	(4,2)	(4,3)	(4,3), (3,2), (2,2), (1,2)
7	(4,3)	(3,3)	(3,3), (3,2), (2,2), (1,2)
8	(3,3)	(3,4), (2,3)	(3,4), (2,3), (3,2), (2,2), (1,2)
9	(3,4)	(3,5), (2,4)	(3,5), (2,4), (2,3), (3,2), (2,2), (1,2)
10	(3,5)	(4,5)	(4,5), (2,4), (2,3), (3,2), (2,2), (1,2)
11	(4,5)	(5,5)	(5,5), (2,4), (2,3), (3,2), (2,2), (1,2)
12	(5,5)	GOAL REACHED	(2,4), (2,3), (3,2), (2,2), (1,2)

DFS path : (1,1), (2,1), (3,1), (4,1), (4,2), (4,3), (3,3), (3,4), (3,5), (4,5), (5,5)

DFS moves : Down, Down, Down, Right, Right, Up, Right, Right, Down, Down

DFS cost : 10 expanded: 12 peak: 6

CONCLUSION:

The warehouse delivery robot was treated as a goal-based intelligent agent. Its PEAS description was written out, and the task was posed as a search problem with the initial state (1,1), the goal state (5,5), four movement actions, a transition model and a uniform step cost of 1.

Breadth-First Search and Depth-First Search were then run with the same neighbour order (Down, Right, Up, Left). BFS found the route $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$ with a cost of 8, which equals the Manhattan-distance lower bound and is therefore optimal. DFS reached the goal with a cost of 10 by way of the route $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$.

BFS expanded 18 nodes and DFS expanded 12, but fewer expansions did not give DFS a better path. BFS is complete and optimal when all step costs are equal, so it is the more suitable algorithm here, and its larger memory use is not an issue on such a small grid. A goal-based agent that plans with BFS gives the robot a dependable shortest-path delivery route.